Практическая работа №7

Мини-проект «Список задач» (итоговая работа)

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Собрать полноценное веб-приложение «Список задач» с полным набором CRUD-операций через браузер, защищёнными соединениями, параметрами в URL и паттерном PRG.

Технологии: Python 3; Flask (маршруты с <int:task_id>, request, redirect, url_for); Jinja2; SQLite (CASE WHEN, rowcount); HTML, CSS, JavaScript confirm().

Реализация: Работа с базой вынесена в отдельные функции, маршруты Flask только вызывают их и делают редирект. Действия, меняющие данные, выполняются только POST-формами, поэтому GET-запрос на /delete/1 возвращает 405. Конвертер <int:task_id> сам проверяет, что id — число (иначе 404). Статус переключается одним запросом UPDATE … SET status = CASE WHEN status = 'done' THEN 'new' ELSE 'done' END. Если rowcount = 0, выводится «Задача #N не найдена».

Основные модули:

- `src/app.py` — слой работы с базой: fetch_all_tasks() (R), insert_task() (C), toggle_task_status() (U), delete_task_by_id() (D); каждая функция использует try/except/finally. Маршруты: / (GET, POST), /toggle/<int:task_id> (POST), /delete/<int:task_id> (POST).
- `src/templates/index.html` — форма добавления, список задач с кнопками ✅/↩️ (переключение статуса) и 🗑️ (удаление с подтверждением), вывод ошибок.

Ход работы

Шаг 4. Запуск и базовый сценарий

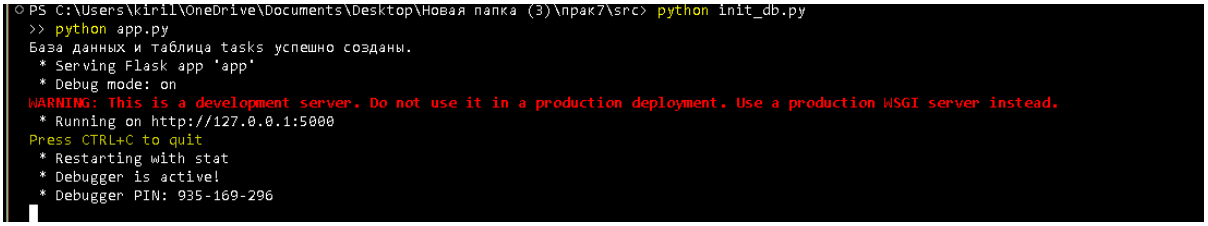


Рисунок 1 — запуск: python init_db.py и python app.py, сервер работает на http://127.0.0.1:5000

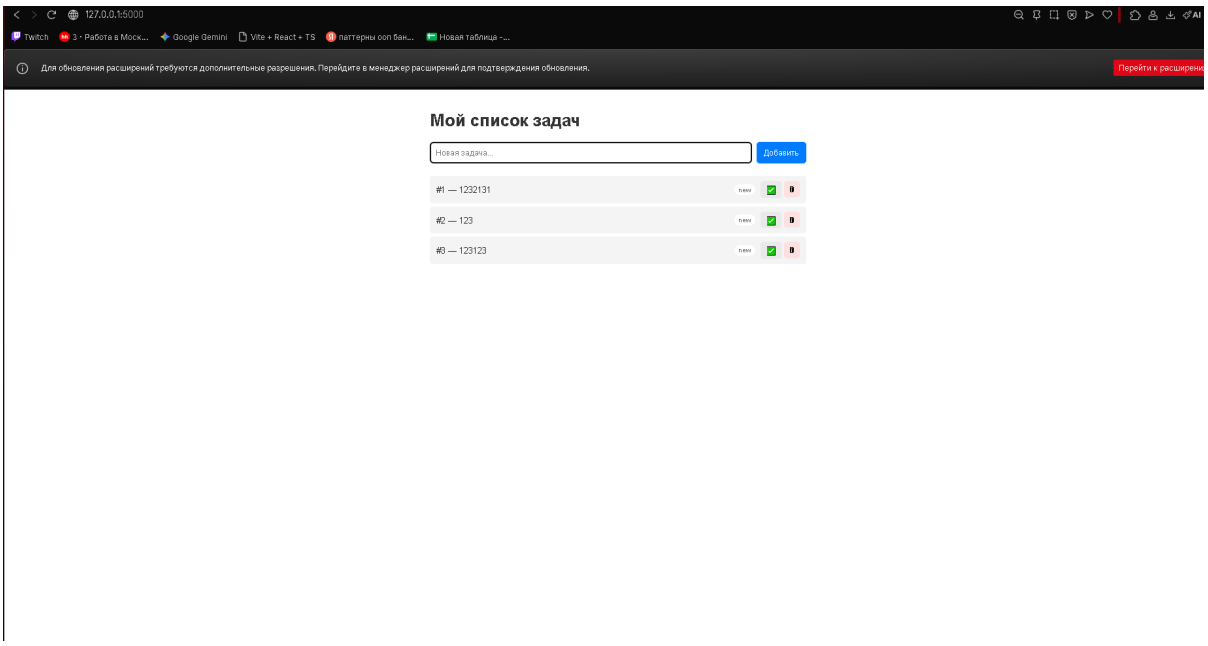


Рисунок 2 — список задач со статусом и кнопками «выполнено» (переключить статус) и «удалить»

Шаг 4. Удаление с подтверждением

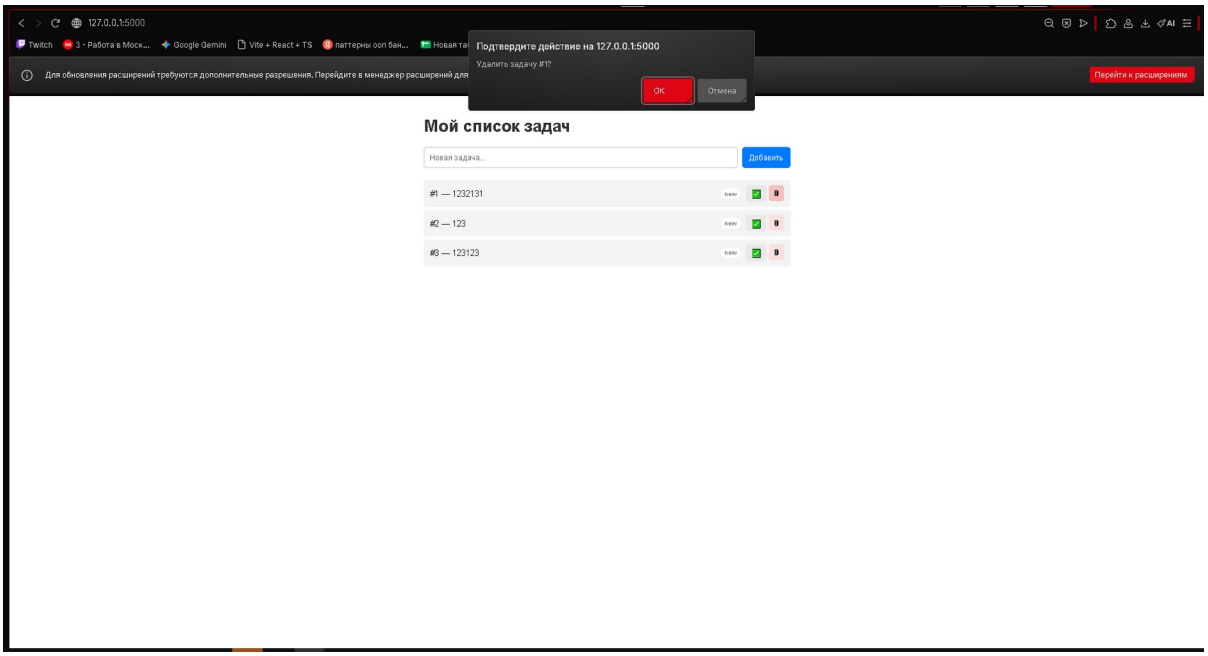


Рисунок 3 — нажатие кнопки «удалить» — браузер запрашивает подтверждение «Удалить задачу #1?»

Шаг 6.1. GET на действие

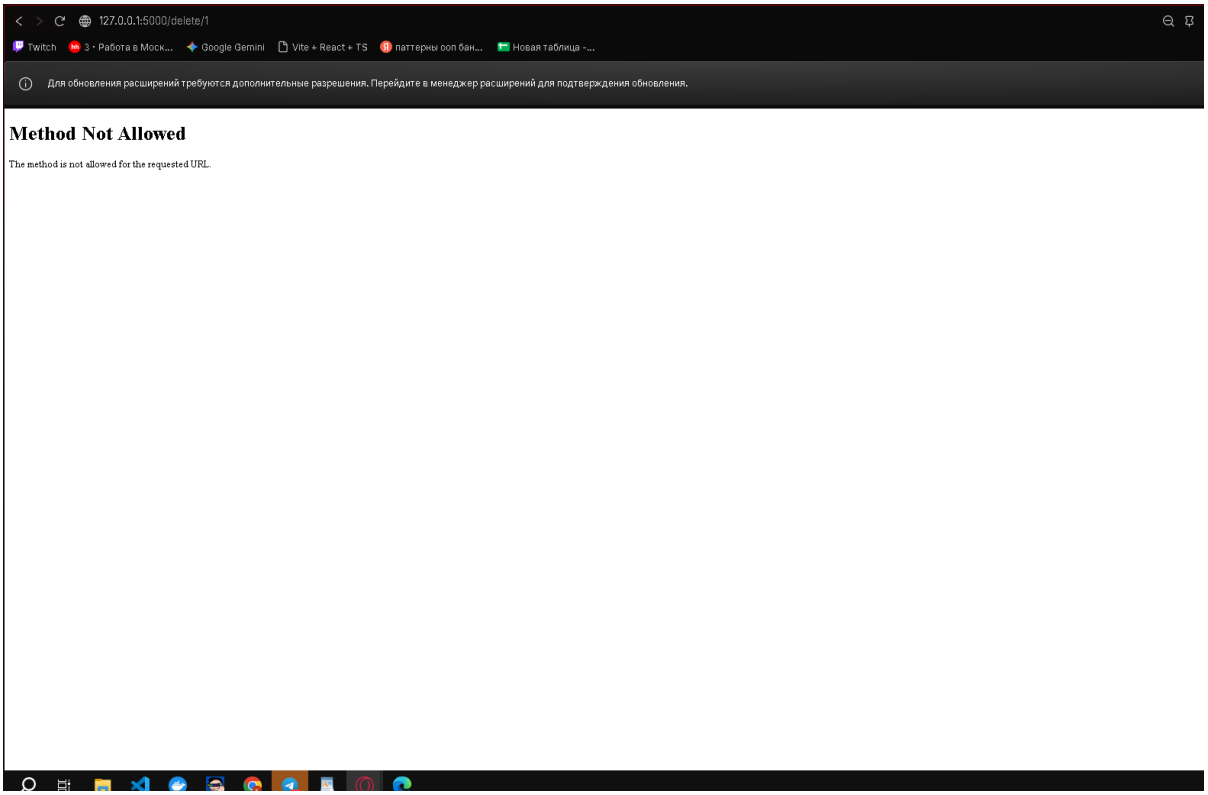


Рисунок 4 — GET-запрос на /delete/1 из адресной строки — ответ 405 Method Not Allowed, задача не удалена

Примеры запросов и ответов

```
POST /  title=Купить хлеб → 302 Found, Location: /
POST /toggle/1          → 302 Found (статус new → done)
POST /delete/2          → 302 Found (задача удалена)
GET  /delete/1          → 405 Method Not Allowed
GET  /toggle/abc        → 404 Not Found
POST /toggle/9999       → 200 OK, «Задача #9999 не найдена.»
```

Ответы на контрольные вопросы

1. **Почему «переключить статус» и «удалить» отправляются POST-формой, а не ссылкой?**

Ссылка выполняет GET, а GET не должен менять данные: его может повторить браузер, кэш или случайный переход. Изменяющие действия выполняются только POST.

2. **Что делает конвертер <int:task_id> и почему не нужен isdigit()?**

Берёт часть URL, проверяет, что это целое число, преобразует в int и передаёт в функцию. Если в URL не число — Flask сам возвращает 404.

3. **Что означает rowcount = 0 после UPDATE или DELETE?**

Ни одна строка не была изменена — задачи с таким id нет.

4. **Почему каждая функция открывает своё соединение?**

Функция самостоятельна: сама открывает, использует и гарантированно закрывает соединение. Это исключает утечки и блокировки и упрощает код маршрутов.

5. **Где в toggle_task_status находится commit() и зачем он?**

Внутри try сразу после execute. Без него изменение статуса не сохранится в файле базы и пропадёт после перезапуска.

6. **Что будет, если открыть /delete/3 в адресной строке?**

Браузер отправит GET, а маршрут принимает только POST — ответ 405 Method Not Allowed, задача не удалится.

7. **Зачем onsubmit="return confirm(...)" и можно ли обойти эту защиту?**

Это подтверждение в браузере от случайного удаления. Обойти можно (отключить JavaScript или отправить POST напрямую), поэтому это защита от ошибок пользователя, а не мера безопасности.

8. **Почему return redirect(...) стоит после try/finally?**

Редирект должен выполняться только после успешной операции и закрытия соединения. При ошибке функция выходит раньше — из except с сообщением.

Вывод

Создано итоговое веб-приложение «Список задач» с полным CRUD через браузер. Применены параметризованные запросы, транзакции с commit/rollback, гарантированное закрытие соединений, паттерн PRG, параметры URL и проверка некорректных запросов (405, 404, несуществующий id).